

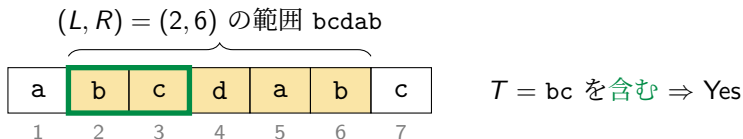
AtCoder ABC477 – C

Range Search Query の解説（尺取り法）

AtCoder Beginner Contest 477 / 300 点

問題の概要

文字列 S, T と Q 個のクエリ (L, R) . S の L 文字目から R 文字目の中に T が連続して現れるかを Yes / No で答える.



サンプル 1: $S = abcdabc$, $T = bc$. $(3, 5)$ は cda なので No, $(3, 7)$ は $cdabc$ なので Yes.

制約

$|S| \leq 4 \times 10^5$, $|T| \leq 10$, $Q \leq 2 \times 10^5$. 入力は Q, S, T の順 (Q が先頭).

まず制約を読む

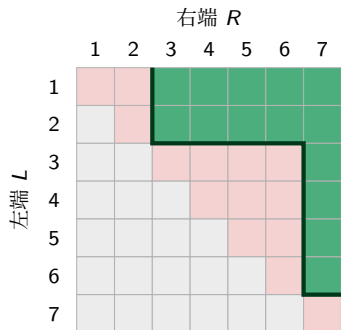
- クエリごとに切り出して探す: $O(QN) = 2 \times 10^5 \times 4 \times 10^5 = 8 \times 10^{10}$ 間に合わない
- 全区間の答えを表にする: $N^2 = 1.6 \times 10^{11}$ マス 確保できない

欲しいもの

前計算で $O(N)$ 程度の大きさの配列を作り、各クエリに $O(1)$ で答えたい.

⇒ 2 次元の「区間 → Yes/No」表を 1 次元に畳める性質を探す.

核心 - 区間を広げても「含む」は崩れない



サンプル 1. 緑 = 含む, 赤 = 含まない

$[l, r]$ が T を含むなら, 広げた $[l, r+1]$ も含む.

⇒ 各行は No...No Yes...Yes. 境目を

$f(l) = T$ を含む最小の右端

と置けば, 答えは $R \geq f(L)$ かどうか.

さらに l を右へずらすと区間は縮むので

$f(l)$ は l について単調非減少

図の境目が右下へ進む階段になっている.

尺取り法 – 右端 r を戻さずに $f(l)$ を全部求める

窓 $s[l:r]$ (半开区間) を持ち, 含まない間は r を進める $\rightarrow$ 含んだら記録 $\rightarrow l$ を進める.

	a	b	c	d	a	b	c	
位置	0	1	2	3	4	5	6	7



$l=0$: r を 3 まで進める $\rightarrow f = 3$



$l=1$: 含んだまま $\rightarrow f = 3$



$l=2$: r を 7 まで進める $\rightarrow f = 7$

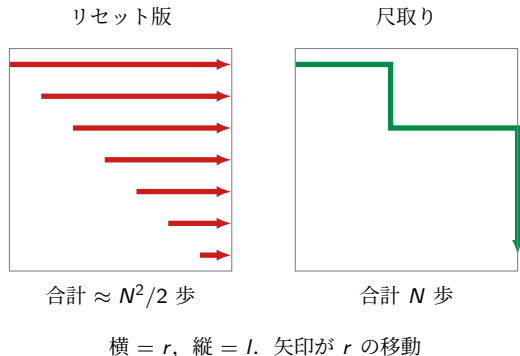


$l=6$: $r = N$ で伸ばせない $\rightarrow$ 番兵

なぜ r を戻さなくてよいか

f は単調非減少なので, 次の l の境目は**今の r 以上**にある. よって r は全体を通して $0 \rightarrow N$ へ**一方向に N 回**しか進まない.

main.py との違い - r をリセットすると二重ループ



N	リセット版	尺取り
1000	500500	1000
2000	2001000	2000
4000	8002000	4000

r の移動回数の実測 ($S = a \times N$, $T = a$)

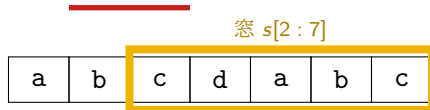
main.py は l ごとに $r = l + \text{len}(t) - 1$ と戻している. さらに判定が `t in s[l:r+1]` で毎回 $O(N)$.

実行時間 (出力除去版) : $N = 2000$ で 0.43 秒,
4000 で 1.72 秒 $\Rightarrow$ 2 倍で約 4 倍の $O(N^2)$

窓が T を含むかは差分で更新する

窓が動くたびに t in 窓 で調べると 1 回 $O(N)$ で台無し. 代わりに窓に完全に収まる T の出現数 cnt を持つ.

その前, l を $1 \rightarrow 2$ に進めたとき: $l=1$ から始まる bc を外す $\Rightarrow \text{cnt} -= 1$



r を進めて右端で終わる $bc \Rightarrow \text{cnt} += 1$

- r を 1 進める: $s[r - |T| : r] = T$ なら $\text{cnt} += 1$
- l を 1 進める: $s[l : l + |T|] = T$ かつ窓に入っていれば $\text{cnt} -= 1$

比較は長さ $|T| \leq 10$ なので定数時間. 前計算は全体で $O(N|T|)$.

擬似コードと実装の要点

```
f = [N+1] * N ; cnt = 0 ; r = 0          # 窓は s[l:r]
for l in 0..N-1:
    while r < N and cnt == 0:             # 含むまで右端を伸ばす
        r += 1
        if r-M >= l and s[r-M:r] == T: cnt += 1
    if cnt > 0: f[l] = r                   # r はそのまま 1-indexed の右端
    if l+M <= r and s[l:l+M] == T: cnt -= 1 # 左端の出現を外す
query (L, R): Yes if f[L-1] <= R
```

- r の初期化が l のループの外にあることが尺取り法の本体.
- 半开区間 $s[l:r]$ で持つと r がそのまま 1-indexed の右端 $\Rightarrow f[L-1] \leq R$ で比べられる.
- $N+1$ は「どの $R \leq N$ でも足りない」番兵.
- 出力は最大 2×10^5 行. `sys.stdin.readline` と `"\n".join` でまとめて出す.

サンプル 1 を手で追う ($S = \text{abcdabc}$, $T = \text{bc}$)

l	r の動き	窓 $s[l:r]$	$f[l]$	l を外すとき
0	$0 \rightarrow 1 \rightarrow 2 \rightarrow 3$	abc	3	ab: 外さない
1	動かない	bc	3	bc: 外す $\rightarrow$ cnt 0
2	$3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$	cdabc	7	cd: 外さない
3, 4	動かない	dabc, abc	7	外さない
5	動かない	bc	7	bc: 外す $\rightarrow$ cnt 0
6	$r = N$ で伸ばせない	c	8 (番兵)	—

r は $0 \rightarrow 7$ の合計 7 回しか進んでいない. $f = [3, 3, 7, 7, 7, 7, 8]$.

(L, R)	$f[L-1]$	判定	答え
(2, 6)	3	$6 \geq 3$	Yes
(3, 5)	7	$5 < 7$	No
(3, 7)	7	$7 \geq 7$	Yes

一段深い見方

- cnt は 0 か 1 にしかない. r を伸ばすのは $\text{cnt} = 0$ の間だけで, 1 になった瞬間に止まる. 個数で持っておくと「 k 個以上含む」にそのまま拡張できる.
- $f(l)$ は「 l 以降で最初に T が始まる位置 $+|T|$ 」と同じ. `ac.py` はこれを後ろから 1 回の走査で直接求めている (計算量は同じ).
- 公式解説は累積和. 各位置で T が始まるかの 0/1 配列の累積和で, 始点が $[L, R - |T| + 1]$ にある出現が 1 つ以上あるかを $O(1)$ で判定する.

尺取り法が使える条件

「区間が条件を満たすなら, それを広げた区間も満たす」という単調性. これがあると境目 $f(l)$ が単調に動き, 右端を戻さずに済む.

- ① 「広げても条件を満たし続ける」なら、左端ごとの境目 $f(l)$ は単調。2 次元の「区間 → 答え」表は 1 次元の f に畳める。
- ② 尺取り法が $O(N)$ なのは右端を戻さないから。 l のループの中で r を初期化したら、それは尺取りではなく二重ループ。
- ③ 窓の状態は差分で更新する。窓が動くたびに in や sum で全体を見直すと $O(N)$ が掛かる。

同系統の問題: ABC032 C – 列の積, ABC130 D – Enough Array, ABC098 D – Xor Sum 2